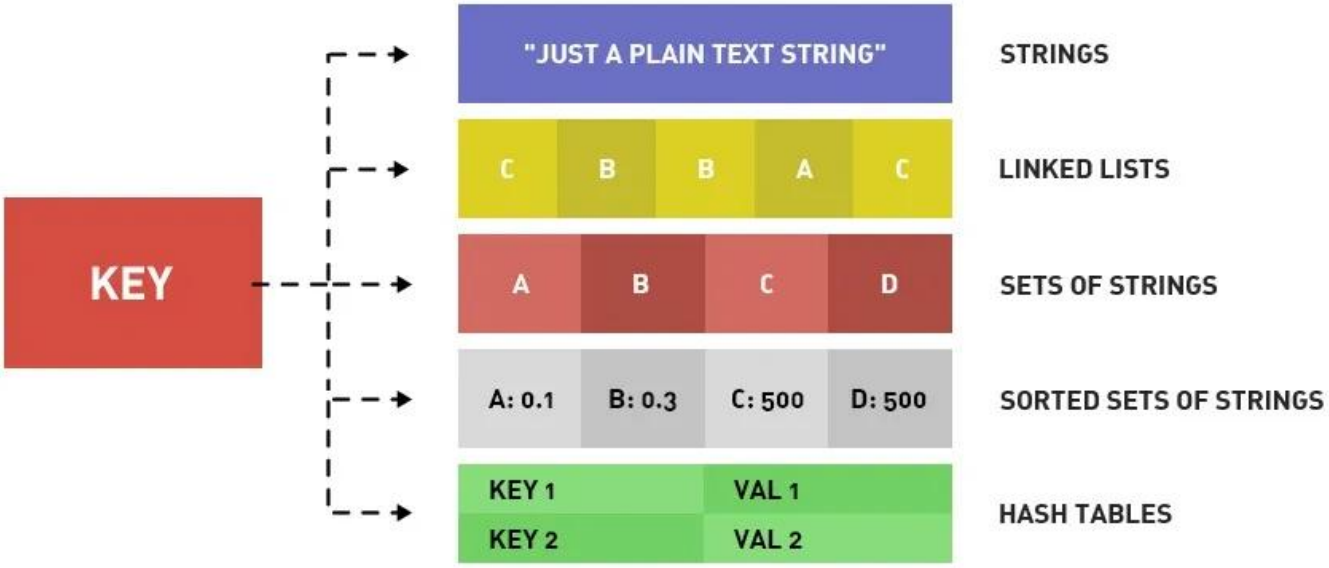
Redis 面试题 - 数据结构

数据结构

讲一下 Redis 底层的数据结构

Redis 提供了丰富的数据类型，常见的有五种数据类型：**String（字符串），Hash（哈希），List（列表），Set（集合）、Zset（有序集合）。**



结构类型	结构存储的值	结构的读写能力
String 字符串	可以是字符串、整数或浮点数	对整个字符串或字符串的一部分进行操作；对整数或浮点数进行自增或自减操作；
List列表	一个链表，链表上的每个节点都包含一个字符串	对链表的两端进行push和pop操作，读取单个或多个元素；根据值查找或删除元素；
Set集合	包含字符串的无序集合	字符串的集合，包含基础的方法有看是否存在添加、获取、删除；还包含计算交集、并集、差集等
Hash散列	包含键值对的无序散列表	包含方法有添加、获取、删除单个元素
Zset有序集合	和散列一样，用于存储键值对	字符串成员与浮点数分数之间的有序映射；元素的排列顺序由分数的大小决定；包含方法有添加、获取、删除单个元素以及根据分值范围或成员来获取元素

随着 Redis 版本的更新，后面又支持了四种数据类型：**BitMap（2.2 版新增）、HyperLogLog（2.8 版新增）、GEO（3.2 版新增）、Stream（5.0 版新增）**。Redis 五种数据类型的应用场景：

- String 类型的应用场景：缓存对象、常规计数、分布式锁、共享 session 信息等。
- List 类型的应用场景：消息队列（但是有两个问题：1. 生产者需要自行实现全局唯一 ID；2. 不能以消费组形式消费数据）等。
- Hash 类型：缓存对象、购物车等。
- Set 类型：聚合计算（并集、交集、差集）场景，比如点赞、共同关注、抽奖活动等。
- Zset 类型：排序场景，比如排行榜、电话和姓名排序等。

Redis 后续版本又支持四种数据类型，它们的应用场景如下：

- BitMap（2.2 版新增）：二值状态统计的场景，比如签到、判断用户登陆状态、连续签到用户总数等；
- HyperLogLog（2.8 版新增）：海量数据基数统计的场景，比如百万级网页 UV 计数等；
- GEO（3.2 版新增）：存储地理位置信息的场景，比如滴滴叫车；
- Stream（5.0 版新增）：消息队列，相比于基于 List 类型实现的消息队列，有这两个特有的特性：自动生成全局唯一消息 ID，支持以消费组形式消费数据。

ZSet 用过吗

用过 zset 实现排行榜的功能。

以博文点赞排名为例，小林发表了五篇博文，分别获得赞为 200、40、100、50、150。

```
# arcticle:1 文章获得了 200 个赞
> ZADD user:xiaolin:ranking 200 arcticle:1
(integer) 1
# arcticle:2 文章获得了 40 个赞
```

```
> ZADD user:xiaolin:ranking 40 arcticle:2
(integer) 1
# arcticle:3 文章获得了 100 个赞
> ZADD user:xiaolin:ranking 100 arcticle:3
(integer) 1
# arcticle:4 文章获得了 50 个赞
> ZADD user:xiaolin:ranking 50 arcticle:4
(integer) 1
# arcticle:5 文章获得了 150 个赞
> ZADD user:xiaolin:ranking 150 arcticle:5
(integer) 1
```

文章 arcticle:4 新增一个赞，可以使用 ZINCRBY 命令（为有序集合 key 中元素 member 的分值加上 increment）：

```
> ZINCRBY user:xiaolin:ranking 1 arcticle:4
"51"
```

查看某篇文章的赞数，可以使用 ZSCORE 命令（返回有序集合 key 中元素个数）：

```
> ZSCORE user:xiaolin:ranking arcticle:4
"50"
```

获取小林文章赞数最多的 3 篇文章，可以使用 ZREVRANGE 命令（倒序获取有序集合 key 从 start 下标到 stop 下标的元素）：

```
# WITHSCORES 表示把 score 也显示出来
> ZREVRANGE user:xiaolin:ranking 0 2 WITHSCORES
1) "arcticle:1"
2) "200"
3) "arcticle:5"
4) "150"
5) "arcticle:3"
```

```
6) "100"
```

获取小林 100 赞到 200 赞的文章，可以使用 ZRANGEBYSCORE 命令（返回有序集合中指定分数区间内的成员，分数由低到高排序）：

```
> ZRANGEBYSCORE user:xiaolin:ranking 100 200 WITHSCORES
1) "arcticle:3"
2) "100"
3) "arcticle:5"
4) "150"
5) "arcticle:1"
6) "200"
```

Redis 中 set 和 zset 区别是什么?

Redis 中的 Set 和 ZSet（Sorted Set，有序集合）都是用于存储多个元素的集合类型，但它们的核心区别在于是否对元素进行排序以及排序方式，这直接影响了它们的适用场景。

Set 是**无序、唯一元素**的集合，适合存储不重复且无需排序的数据（如用户 ID 列表、标签集合）。常用命令：

- 添加元素：`SADD set1 "a" "b" "c"`（向 set1 中添加 3 个元素，重复添加会自动去重）
- 查看所有元素：`SMEMBERS set1`（返回结果无序，如 ["b", "a", "c"]）
- 判断元素是否存在：`SISMEMBER set1 "a"`（存在返回 1，否则 0）
- 集合运算：`SINTER set1 set2`（求两个集合的交集）、`SUNION set1 set2`（求并集）

ZSet 是**有序、唯一元素**的集合，每个元素关联一个 “分数（score）”，并按分数从小到大排序（元素唯一，但分数可重复），适合需要排序或排名的场景（如排行榜、带权重的任务）。常用命令：

- 添加元素（带分数）：`ZADD zset1 10 "a" 20 "b" 15 "c"`（向 zset1 中添加元素，分数分别为 10、20、15）
- 按排名范围获取元素（从小到大）：`ZRANGE zset1 0 -1 WITHSCORES`（返回所有元素及分数，结果按分数排序："a" 10, "c" 15, "b" 20）
- 按分数范围获取元素：`ZRANGEBYSCORE zset1 12 25`（返回分数 12-25 之间的元素："c", "b"）
- 增减元素分数：`ZINCRBY zset1 5 "a"`（将 "a" 的分数 +5，变为 15）
- 获取元素排名：`ZRANK zset1 "b"`（返回 "b" 的排名，从 0 开始，此处返回 2）

具体来说，ZSet 相比 Set 多了很多基于分数的操作：比如根据分数范围获取元素（zrangebyscore）、根据排名获取元素（zrange）、计算元素的排名（zrank）、给元素的分数增减（zincrby）等。而 Set 没有分数概念，只能按元素本身进行操作，无法直接获取 “第 N 个元素” 或 “某个范围内的元素”。

简单来说，核心区别总结：

- 有序性：Set 完全无序，ZSet 通过分数维持有序
- 操作特性：ZSet 多了基于分数的排序、范围查询、排名计算等命令，Set 只有集合本身的交并差等操作
- 适用场景：Set 用于去重且无需排序的场景，ZSet 用于需要排序、排名或权重相关的场景

Zset 底层是怎么实现的?

Zset 类型的底层数据结构是由**压缩列表或跳表**实现的：

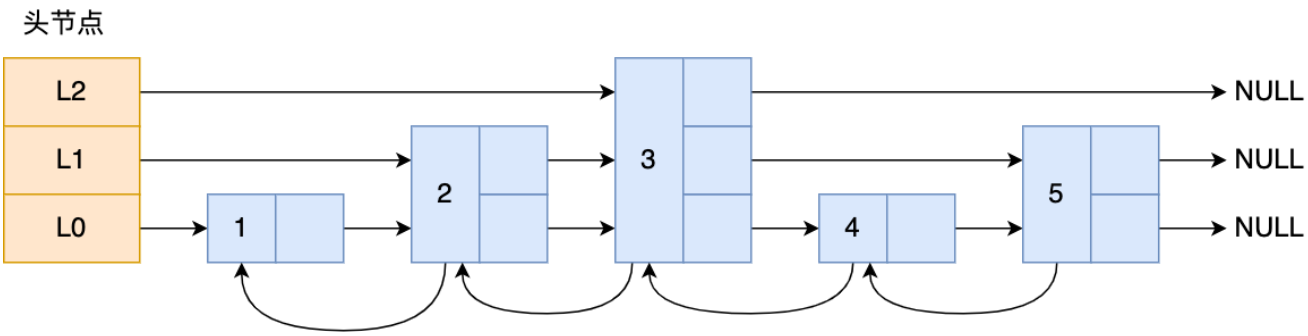
- 如果有序集合的元素个数小于 128 个，并且每个元素的值小于 64 字节时，Redis 会使用**压缩列表**作为 Zset 类型的底层数据结构；
- 如果有序集合的元素不满足上面的条件，Redis 会使用**跳表**作为 Zset 类型的底层数据结构；

在 Redis 7.0 中，压缩列表数据结构已经废弃了，交由 listpack 数据结构来实现了。

跳表是怎么实现的?

链表在查找元素的时候，因为需要逐一查找，所以查询效率非常低，时间复杂度是 O(N)，于是就出现了跳表。**跳表是在链表基础上改进过来的，实现了一种「多层」的有序链表**，这样的好处是能快读定位数据。

那跳表长什么样呢？我这里举个例子，下图展示了一个层级为 3 的跳表。



图中头节点有 L0~L2 三个头指针，分别指向了不同层级的节点，然后每个层级的节点都通过指针连接起来：

- L0 层级共有 5 个节点，分别是节点 1、2、3、4、5；
- L1 层级共有 3 个节点，分别是节点 2、3、5；
- L2 层级只有 1 个节点，也就是节点 3 。

如果我们要在链表中查找节点 4 这个元素，只能从头开始遍历链表，需要查找 4 次，而使用了跳表后，只需要查找 2 次就能定位到节点 4，因为可以在头节点直接从 L2 层级跳到节点 3，然后再往前遍历找到节点 4。

可以看到，这个查找过程就是在多个层级上跳来跳去，最后定位到元素。当数据量很大时，跳表的查找复杂度就是 O(logN)。

那跳表节点是怎么实现多层级的呢？这就需要看「跳表节点」的数据结构了，如下：

```
typedef struct zskiplistNode {
    //Zset 对象的元素值
    sds ele;
    //元素权重值
    double score;
    //后向指针
    struct zskiplistNode *backward;

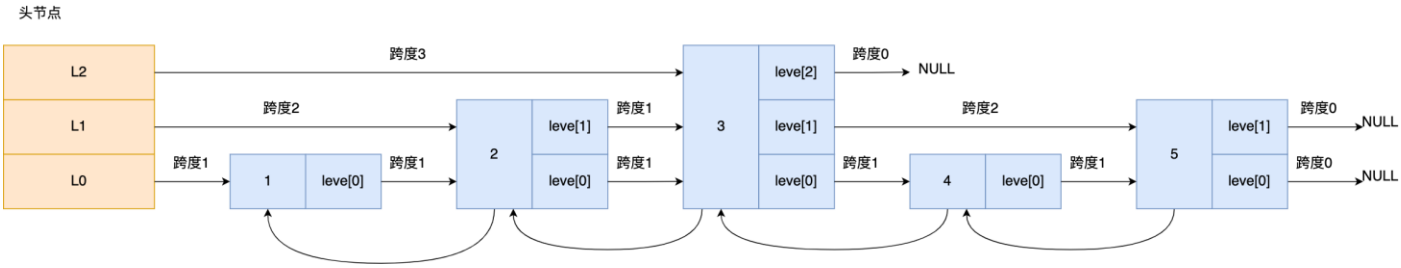
    //节点的 level 数组，保存每层上的前向指针和跨度
    struct zskiplistLevel {
        struct zskiplistNode *forward;
        unsigned long span;
    } level[];
} zskiplistNode;
```

Zset 对象要同时保存「元素」和「元素的权重」，对应到跳表节点结构里就是 sds 类型的 ele 变量和 double 类型的 score 变量。每个跳表节点都有一个后向指针（struct zskiplistNode *backward），指向前一个节点，目的是为了方便从跳表的尾节点开始访问节点，这样倒序查找时很方便。

跳表是一个带有层级关系的链表，而且每一层级可以包含多个节点，每一个节点通过指针连接起来，实现这一特性就是靠跳表节点结构体中的 **zskiplistLevel 结构体类型的 level 数组。**

level 数组中的每一个元素代表跳表的一层，也就是由 zskiplistLevel 结构体表示，比如 leve[0] 就表示第一层，leve[1] 就表示第二层。zskiplistLevel 结构体里定义了「指向下一个跳表节点的指针」和「跨度」，跨度时用来记录两个节点之间的距离。

比如，下面这张图，展示了各个节点的跨度。



第一眼看到跨度的时候，以为是遍历操作有关，实际上并没有任何关系，遍历操作只需要用前向指针（struct zskiplistNode *forward）就可以完成了。

Redis **跳表在创建节点的时候，随机生成每个节点的层数**，并没有严格维持相邻两层的节点数量比例为 2 : 1 的情况。

具体的做法是，**跳表在创建节点时候，会生成范围为[0-1]的一个随机数，如果这个随机数小于 0.25（相当于概率 25%），那么层数就增加 1 层，然后继续生成下一个随机数，直到随机数的结果大于 0.25 结束，最终确定该节点的层数。**

这样的做法，相当于每增加一层的概率不超过 25%，层数越高，概率越低，层高最大限制是 64。

虽然我前面讲解跳表的时候，图中的跳表的「头节点」都是 3 层高，但是其实**如果层高最大限制是 64，那么在创建跳表「头节点」的时候，就会直接创建 64 层高的头节点。**

跳表是怎么设置层高的?

跳表在创建节点时候，会生成范围为[0-1]的一个随机数，如果这个随机数小于 0.25（相当于概率 25%），那么层数就增加 1 层，然后继续生成下一个随机数，直到随机数的结果大于 0.25 结束，最终确定该节点的层数。

Redis 为什么使用跳表而不是用 B+树?

核心原因可以总结为以下几点：

特性	跳表 (Skip List)	B+ 树 (B+ Tree)
主要应用场景	内存数据库 (Redis)	磁盘数据库 (MySQL)
查找复杂度	平均 O(log N)	O(log N) (底数大，高度低)
实现难度	低 (几十到几百行)	高 (需处理分裂、合并)
写操作代价	局部指针修改，无全局重平衡	可能引发连锁分裂/合并

1. 内存 vs. 磁盘的设计初衷 (最重要的原因)：

- **B+ 树是为磁盘 I/O 优化的：** B+ 树的设计核心是降低树的高度，从而减少磁盘 I/O 次数。
- **Redis 是纯内存操作：** 在内存中，没有“磁盘 I/O”的瓶颈。内存中指针跳转的速度非常快。跳表虽然比 B+ 树高（层数多），但在内存中多几次指针跳转的开销非常小。因此，B+ 树为了减少高度而设计的复杂页管理机制，在内存场景下显得多余且笨重。

2、实现复杂度与代码维护：

- 跳表（Skip List）：本质上是“多层链表”。其插入、删除逻辑主要是修改指针，代码实现非常简洁（几百行 C 代码）。
- B+ 树：插入和删除可能引发节点的分裂（Split）和合并（Merge），甚至需要对整棵树进行重平衡。实现一个健壮、高效的 B+ 树非常复杂，代码量大且难以调试。
- **结论：** Redis 作者 Antirez 非常看重代码的可读性和简洁性，跳表以更低的代码复杂度达到了同等级别的性能 O(log N)。

3. 写入性能与重平衡代价：

- **B+ 树的写入抖动：** 当插入数据导致页分裂时，可能需要移动大量数据或改变树结构，这会产生性能抖动。
- **跳表的局部性：** 跳表的插入和删除操作是**局部**的。插入一个节点只需要修改前后节点的指针，并根据概率随机生成层高。它不需要像红黑树或 B+ 树那样进行全局的旋转或复杂的结构调整。

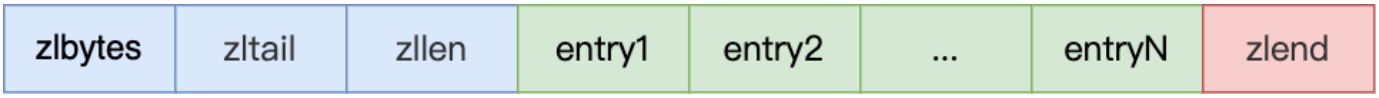
Redis 选跳表而非 B+ 树，不是因为跳表 “更先进”，而是**场景匹配度更高**：

1. 内存场景下，B+ 树的 IO 优化优势无用武之地；
2. 高频更新场景中，跳表的实现简单性、性能稳定性更契合 Redis 需求；
3. zset 的核心操作（增删改查、范围查询），跳表能以更低的复杂度实现同等甚至更优的性能。

简单说：B+ 树是 “磁盘场景的最优解”，而跳表是 “Redis 内存有序数据场景的最优解”，技术选择的核心是 “适配场景”。

压缩列表是怎么实现的?

压缩列表是 Redis 为了节约内存而开发的，它是**由连续内存块组成的顺序型数据结构**，有点类似于数组。



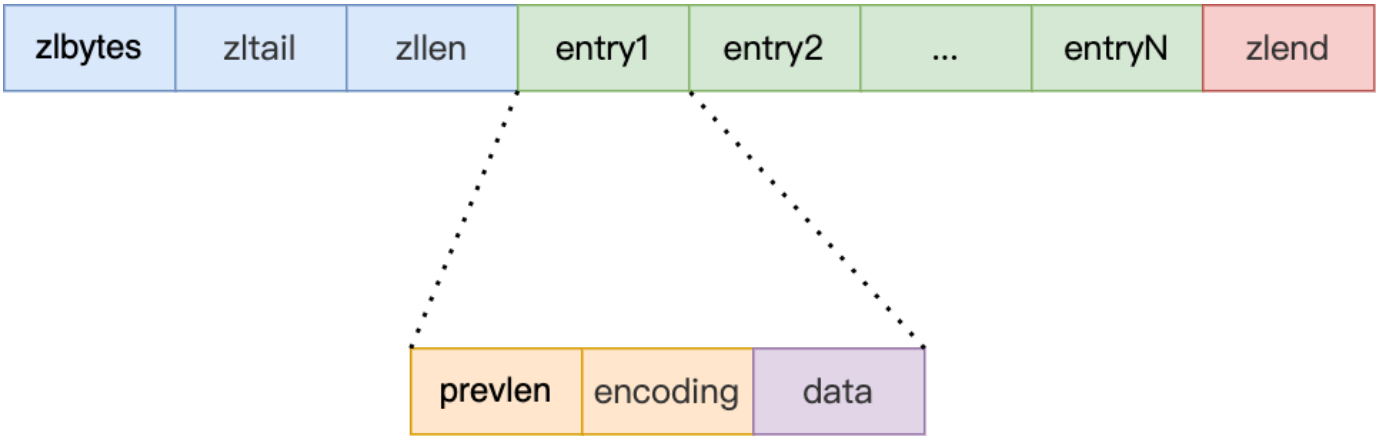
压缩列表在表头有三个字段：

- ***zlbytes***，记录整个压缩列表占用对内存字节数；
- ***zltail***，记录压缩列表「尾部」节点距离起始地址由多少字节，也就是列表尾的偏移量；
- ***zllen***，记录压缩列表包含的节点数量；

- ***zlend***，标记压缩列表的结束点，固定值 0xFF（十进制 255）。

在压缩列表中，如果我们要查找定位第一个元素和最后一个元素，可以通过表头三个字段（zllen）的长度直接定位，复杂度是 O(1)。而**查找其他元素时，就没有这么高效了，只能逐个查找，此时的复杂度就是 O(N) 了，因此压缩列表不适合保存过多的元素。**

另外，压缩列表节点（entry）的构成如下：



压缩列表节点包含三部分内容：

- **prevlen**，记录了「前一个节点」的长度，目的是为了实现从后向前遍历；
- **encoding**，记录了当前节点实际数据的「类型和长度」，类型主要有两种：字符串和整数。
- **data**，记录了当前节点的实际数据，类型和长度都由 encoding 决定；

当我们往压缩列表中插入数据时，压缩列表就会根据数据类型是字符串还是整数，以及数据的大小，会使用不同空间大小的 prevlen 和 encoding 这两个元素里保存的信息，**这种根据数据大小和类型进行不同的空间大小分配的设计思想，正是 Redis 为了节省内存而采用的。**

压缩列表的缺点是会发生连锁更新的问题，因此**连锁更新一旦发生，就会导致压缩列表占用的内存空间要多次重新分配，这就会直接影响到压缩列表的访问性能。**

所以说，**虽然压缩列表紧凑型的内存布局能节省内存开销，但是如果保存的元素数量增加了，或是元素变大了，会导致内存重新分配，最糟糕的是会有「连锁更新」的问题。**

因此，**压缩列表只会用于保存的节点数量不多的场景**，只要节点数量足够小，即使发生连锁更新，也是能接受的。

虽说如此，Redis 针对压缩列表在设计上的不足，在后来的版本中，新增设计了两种数据结构：quicklist（Redis 3.2 引入） 和 listpack（Redis 5.0 引入）。这两种数据结构的设计目标，就是尽可能地保持压缩列表节省内存的优势，同时解决压缩列表的「连锁更新」的问题。

介绍一下 Redis 中的 listpack

quicklist 虽然通过控制 quicklistNode 结构里的压缩列表的大小或者元素个数，来减少连锁更新带来的性能影响，但是并没有完全解决连锁更新的问题。

因为 quicklistNode 还是用了压缩列表来保存元素，压缩列表连锁更新的问题，来源于它的结构设计，所以要想彻底解决这个问题，需要设计一个新的数据结构。

于是，Redis 在 5.0 新设计一个数据结构叫 listpack，目的是替代压缩列表，它最大特点是 listpack 中每个节点不再包含前一个节点的长度了，压缩列表每个节点正因为需要保存前一个节点的长度字段，就会有连锁更新的隐患。

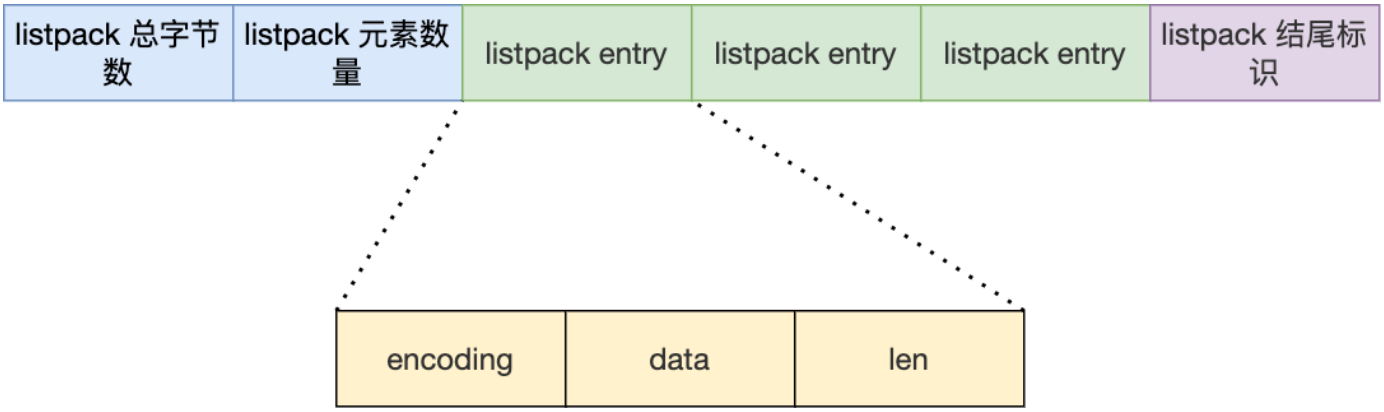
listpack 采用了压缩列表的很多优秀的设计，比如还是用一块连续的内存空间来紧凑地保存数据，并且为了节省内存的开销，listpack 节点会采用不同的编码方式保存不同大小的数据。

我们先看看 listpack 结构：



listpack 头包含两个属性，分别记录了 listpack 总字节数和元素数量，然后 listpack 末尾也有个结尾标识。图中的 listpack entry 就是 listpack 的节点了。

每个 listpack 节点结构如下：



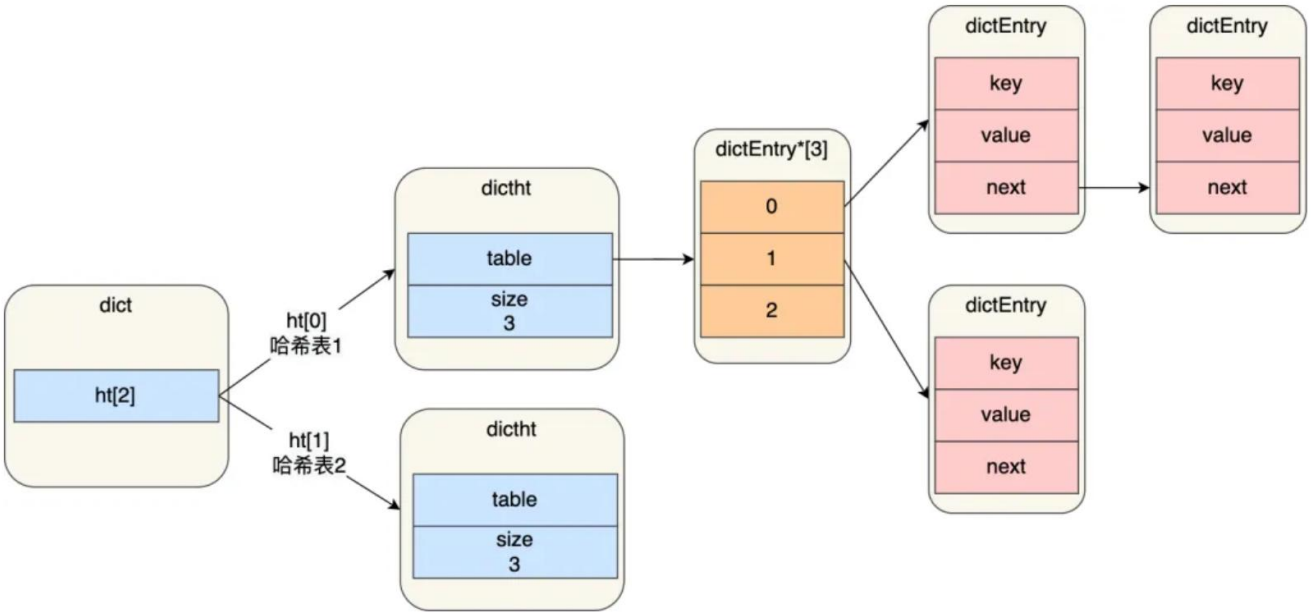
主要包含三个方面内容：

- encoding，定义该元素的编码类型，会对不同长度的整数和字符串进行编码；
- data，实际存放的数据；
- len，encoding+data 的总长度；

可以看到，**listpack 没有压缩列表中记录前一个节点长度的字段了，listpack 只记录当前节点的长度，当我们向 listpack 加入一个新元素的时候，不会影响其他节点的长度字段的变化，从而避免了压缩列表的连锁更新问题。**

哈希表是怎么扩容的？

进行 rehash 的时候，需要用上 2 个哈希表了。

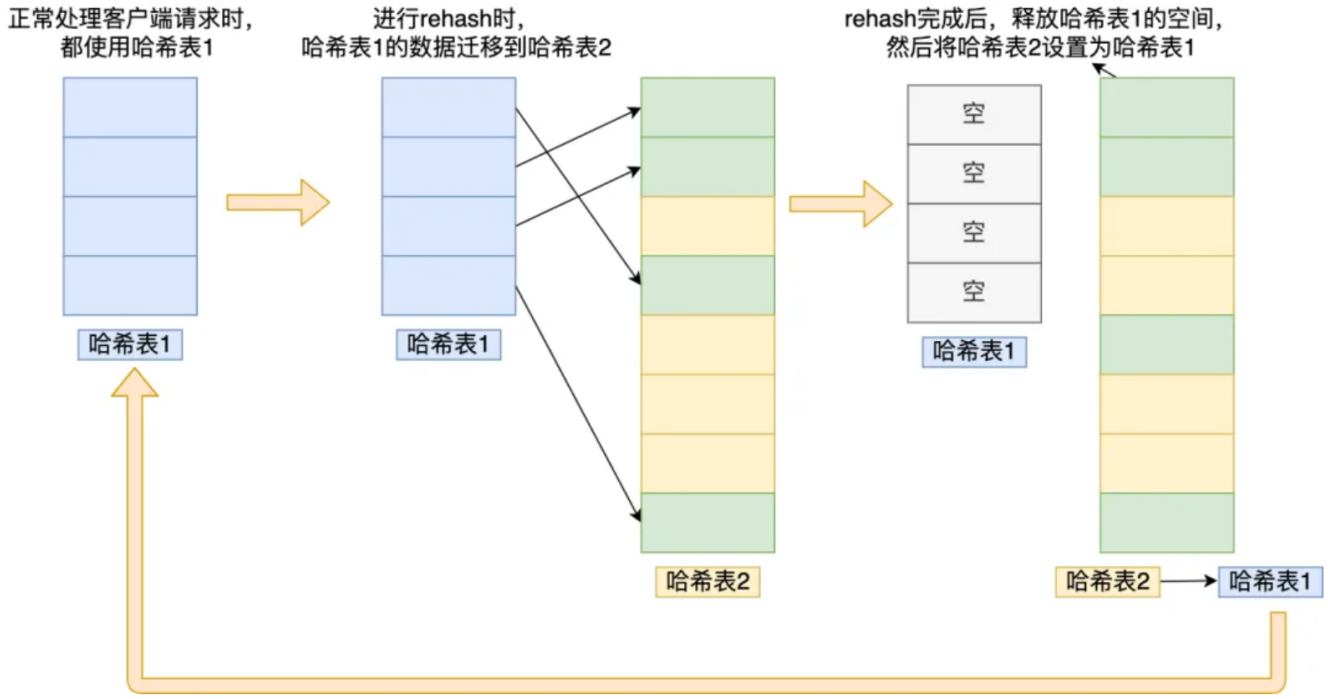


在正常服务请求阶段，插入的数据，都会写入到「哈希表 1」，此时的「哈希表 2 」 并没有被分配空间。

随着数据逐步增多，触发了 rehash 操作，这个过程分为三步：

- 给「哈希表 2」 分配空间，一般会比「哈希表 1」 大 2 倍；
- 将「哈希表 1 」的数据迁移到「哈希表 2」 中；
- 迁移完成后，「哈希表 1 」的空间会被释放，并把「哈希表 2」 设置为「哈希表 1」，然后在「哈希表 2」 新创建一个空白的哈希表，为下次 rehash 做准备。

为了方便你理解，我把 rehash 这三个过程画在了下面这张图：



这个过程看起来简单，但是其实第二步很有问题，**如果「哈希表 1 」的数据量非常大，那么在迁移至「哈希表 2 」的时候，因为会涉及大量的数据拷贝，此时可能会对 Redis 造成阻塞，无法服务其他请求。**

为了避免 rehash 在数据迁移过程中，因拷贝数据的耗时，影响 Redis 性能的情况，所以 Redis 采用了**渐进式 rehash**，也就是将数据的迁移的工作不再是一次性迁移完成，而是分多次迁移。

渐进式 rehash 步骤如下：

- 给「哈希表 2」 分配空间；
- **在 rehash 进行期间，每次哈希表元素进行新增、删除、查找或者更新操作时，Redis 除了会执行对应的操作之外，还会顺序将「哈希表 1 」中索引位置上的所有 key-value 迁移到「哈希表 2」 上**；
- 随着处理客户端发起的哈希表操作请求数量越多，最终在某个时间点会把「哈希表 1 」的所有 key-value 迁移到「哈希表 2」，从而完成 rehash 操作。

这样就巧妙地把一次性大量数据迁移工作的开销，分摊到了多次处理请求的过程中，避免了一次性 rehash 的耗时操作。

在进行渐进式 rehash 的过程中，会有两个哈希表，所以在渐进式 rehash 进行期间，哈希表元素的删除、查找、更新等操作都会在这两个哈希表进行。比如，查找一个 key 的值的话，先会在「哈希表 1」 里面进行查找，如果没找到，就会继续到哈希表 2 里面进行找到。

另外，在渐进式 rehash 进行期间，新增一个 key-value 时，会被保存到「哈希表 2 」里面，而「哈希表 1」 则不再进行任何添加操作，这样保证了「哈希表 1 」的 key-value 数量只会减少，随着 rehash 操作的完成，最终「哈希表 1 」就会变成空表。

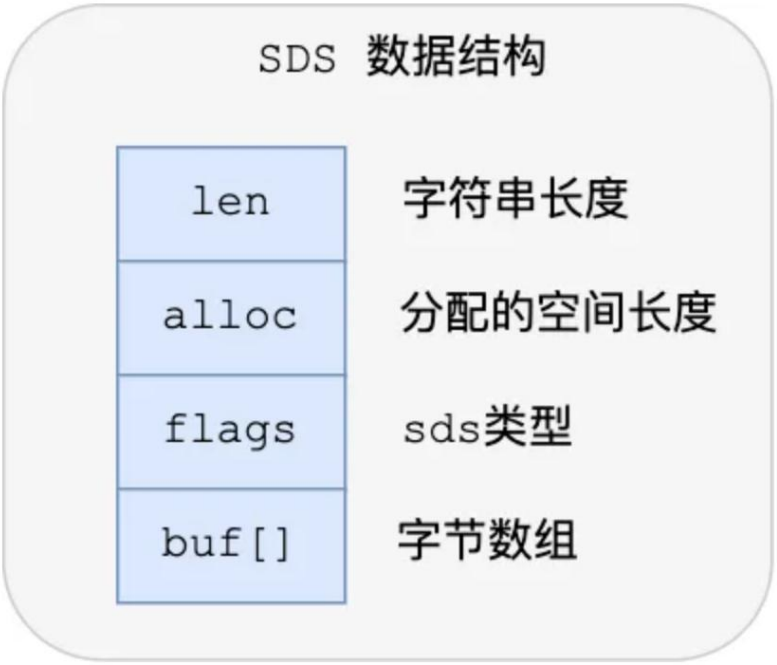
哈希表扩容的时候，有读请求怎么查?

查找一个 key 的值的话，先会在「哈希表 1」 里面进行查找，如果没找到，就会继续到哈希表 2 里面进行找到。

String 是使用什么存储的?为什么不用 c 语言中的字符串?

Redis 的 String 字符串是用 SDS 数据结构存储的。

下图就是 Redis 5.0 的 SDS 的数据结构：



结构中的每个成员变量分别介绍下：

- **len，记录了字符串长度。**这样获取字符串长度的时候，只需要返回这个成员变量值就行，时间复杂度只需要 O（1）。
- **alloc，分配给字符数组的空间长度。**这样在修改字符串的时候，可以通过 `alloc - len` 计算出剩余的空间大小，可以用来判断空间是否满足修改需求，如果不满足的话，就会自动将 SDS 的空间扩展至执行修改所需的大小，然后才执行实际的修改操作，所以使用 SDS 既不需要手动修改 SDS 的空间大小，也不会出现前面所说的缓冲区溢出的问题。
- **flags，用来表示不同类型的 SDS。**一共设计了 5 种类型，分别是 sdshdr5、sdshdr8、sdshdr16、sdshdr32 和 sdshdr64，后面在说明区别之处。
- **buf[]，字符数组，用来保存实际数据。**不仅可以保存字符串，也可以保存二进制数据。

总的来说，Redis 的 SDS 结构在原本字符数组之上，增加了三个元数据：len、alloc、flags，用来解决 C 语言字符串的缺陷。

> O（1）复杂度获取字符串长度

C 语言的字符串长度获取 strlen 函数，需要通过遍历的方式来统计字符串长度，时间复杂度是 O（N）。

而 Redis 的 SDS 结构因为加入了 len 成员变量，那么**获取字符串长度的时候，直接返回这个成员变量的值就行，所以复杂度只有 O（1）。**

> 二进制安全

因为 SDS 不需要用 “\0” 字符来标识字符串结尾了，而是**有个专门的 len 成员变量来记录长度，所以可存储包含 “\0” 的数据。**但是 SDS 为了兼容部分 C 语言标准库的函数， SDS 字符串结尾还是会加上 “\0” 字符。

因此， SDS 的 API 都是以处理二进制的方式来处理 SDS 存放在 buf[] 里的数据，程序不会对其中的数据做任何限制，数据写入的时候时什么样的，它被读取时就是什么样的。

通过使用二进制安全的 SDS，而不是 C 字符串，使得 Redis 不仅可以保存文本数据，也可以保存任意格式的二进制数据。

> 不会发生缓冲区溢出

C 语言的字符串标准库提供的字符串操作函数，大多数（比如 strcat 追加字符串函数）都是不安全的，因为这些函数把缓冲区大小是否满足操作需求的工作交由开发者来保证，程序内部并不会判断缓冲区大小是否足够用，当发生了缓冲区溢出就有可能造成程序异常结束。

所以，Redis 的 SDS 结构里引入了 alloc 和 len 成员变量，这样 SDS API 通过 `alloc - len` 计算，可以算出剩余可用的空间大小，这样在对字符串做修改操作的时候，就可以由程序内部判断缓冲区大小是否足够用。

而且，**当判断出缓冲区大小不够用时，Redis 会自动将扩大 SDS 的空间大小**，以满足修改所需的大小。

redis 的 Zset，在项目里具体用法是什么?

Redis 的 Zset（有序集合）在项目里的核心用法是**需要给元素排序、取排名、按分数范围筛选**的场景，因为它既能像 Set 一样保证元素唯一，又能给每个元素关联一个分数，自动按分数排序，查询效率还高。

最常用的场景是**排行榜**，比如游戏里的玩家积分排名、电商的商品销量榜单、社区的用户贡献值排名。比如做游戏积分榜，把玩家 ID 作为元素，积分作为分数，用 ZADD 命令添加或更新玩家积分，想查前 10 名就用 `ZREVRANGE`（降序取前 10），想知道某个玩家的排名就用 `ZREVRANK`，还能通过 ZSCORE 查他的具体积分，这些操作都是实时的，效率很高。

然后是**延迟任务队列**，比如电商订单 30 分钟未支付自动取消、定时发送通知。这里用任务 ID 作为元素，把 “任务执行时间戳” 作为分数，后台开一个线程不断用 `ZRANGEBYSCORE` 查询 “分数小于当前时间戳” 的任务，执行完就用 ZREM 删掉，这样就能实现定时任务，不用复杂的调度框架。

还有**范围查询场景**，比如查询 “积分在 1000-2000 之间的玩家” “销量前 50 到 100 名的商品”，直接用 `ZRANGEBYSCORE` 或 `ZREVRANGEBYSCORE` 就能快速筛选出来，比在数据库里做排序查询高效得多，尤其是数据量大的时候。

另外，还能用来做**带权重的消息队列**，给重要的消息设置更高的分数，消费者优先处理分数高的消息，保证核心业务的优先级。